

Relatório Técnico: Sistemas Embarcados

Identificação do Aluno	Número USP	Data de Entrega
Giovanni Antonio Silva de Oliveira	14575064	05/06/2026

1. Identificação do Experimento

- **Título:** Atividade 5 - Ultrassom
- **Disciplina:** PSI 3441 - Arquitetura de Sistemas Embarcados
- **Professor:** Gustavo Pamplona Rehder

2. Respostas, Comentários e Análises

A atividade foi feita criando-se a lib `HC_SR04.h`, que usa a interrupção do timer e a geração do pulso pelo mesmo. a lógica de ativação segue gerando a interrupção, e checando o estado do pino para saber se a ultima borda foi de subida ou descida (ele é configurado para gerar a interrupção em ambas as bordas).

3. Código-Fonte

```
////////////////////////////////////  
//////////HC_SR04.h  
////////////////////////////////////  
  
#ifndef HC_SR04_H_  
#define HC_SR04_H_  
  
#include <stdbool.h>  
  
void hc_sr04_init(void);  
  
bool hc_sr04_read(float *distancia);
```

```
#endif /* HC_SR04_H_ */
```

```
////////////////////////////////////
//////////HC_SR04.c
////////////////////////////////////

#include <zephyr.h>
#include <drivers/gpio.h>
#include "pwm_z402.h"
#include "hc_sr04.h"

#define TPM_IRQ_LINE TPM1_IRQn
#define TPM_IRQ_PRIORITY 1
#define PULSE_WIDTH_TICKS 4

// variáveis privadas
static volatile uint16_t capture1 = 0;
static volatile uint16_t capture2 = 0;
static volatile uint16_t pulse_width = 0;
static volatile uint8_t new_data = 0;
static volatile uint8_t esperando_descida = 0;

// A ISR privada
static void tpm1_isr(const void *arg)
{
    TPM1->CONTROLS[0].CnSC |= TPM_CnSC_CHE_MASK;
    TPM1->STATUS = TPM_STATUS_CH0F_MASK; // zerra a flag que gerou a
interrupção

    uint16_t current_capture = TPM1->CONTROLS[0].CnV;
    // Lê fisicamente o bit 20 da porta E (PTE20)
    uint8_t pino_alto = (GPIOE->PDIR & (1 << 20)) ? 1 : 0;

    // borda de Subida (Pino está em 3.3V)
    if (pino_alto)
    {
        capture1 = current_capture;
        esperando_descida = 1;
    }
    // borda Descida (Pino está em 0V)
    else
    {

```

```

        if (esperando_descida == 1)
        {
            capture2 = current_capture;
            pulse_width = (capture2 >= capture1) ? (capture2 -
capture1) : ((65535 - capture1) + capture2 + 1);
            new_data = 1;
            esperando_descida = 0;
        }
    }
}

// FUNÇÕES PÚBLICAS

void hc_sr04_init(void)
{
    // Conecta a interrupção via Zephyr
    IRQ_CONNECT(TPM_IRQ_LINE, TPM_IRQ_PRIORITY, tpm1_isr, NULL, 0);
    irq_enable(TPM_IRQ_LINE);

    // Inicializa TPM1 com módulo e prescaler
    pwm_tpm_Init(TPM1, TPM_PLLFLL, 65535, TPM_CLK, PS_128, EDGE_PWM);

    // Configura TPM1_CH1 como PWM_H
    pwm_tpm_Ch_Init(TPM1, 1, TPM_PWM_H, GPIOE, 21);
    TPM1->CONTROLS[1].CnV = PULSE_WIDTH_TICKS;

    esperando_descida = 0;
    // Configura TPM1_CH0 como input capture em AMBAS as bordas
    pwm_tpm_Ch_Init(TPM1, 0, TPM_INPUT_CAPTURE_BOTH |
TPM_CHANNEL_INTERRUPT, GPIOE, 20);
}

bool hc_sr04_read(float *distancia)
{
    if (new_data)
    {
        new_data = 0;
        *distancia = pulse_width * 0.0458f;
        return true;
    }
    return false;
}

```

```

////////////////////////////////////
////////////////Main.c
////////////////////////////////////

#include <zephyr.h>
#include <stdio.h>
#include "hc_sr04.h"

void main(void)
{
    // Inicializa o hardware do sensor em 1 linha
    hc_sr04_init();

    printf("Sensor distancia conectado.\n");

    while (1)
    {
        float distancia_atual;

        // Passa o endereço da variável para a biblioteca preencher
        if (hc_sr04_read(&distancia_atual))
        {
            int dist_inteira = (int)distancia_atual;
            int dist_decimal = (int)(distancia_atual * 10) % 10;

            printf("Distancia: %d.%d cm\n", dist_inteira,
dist_decimal);
        }

        k_msleep(100);
    }
}

```

4. Referências e Links Externos

- Link do Repositório (GitHub/GitLab):

<https://github.com/Djovanne/PSI3441---Arquitetura-de-Sistemas-Embarcados/tree/main/Atividade%205%20-%20Ultrassom>